

P1227: Signed ssize() functions, unsigned size() functions (Revision 2)

Revision history

- 2018-10-08: R0: Proposal to add ssize to all STL containers as well as standalone std::ssize.
- 2019-01-21: R1: Incorporate P1089R2.
- 2019-02-22: R2: Incorporate Kona LWG feedback.

Introduction.

When span was adopted into C++17, it used a signed integer both as an index and a size. Partly this was to allow for the use of "-1" as a sentinel value to indicate a type whose size was not known at compile time. But having an STL container whose size() function returned a signed value was problematic, so P1089 was introduced to "fix" the problem. It received majority support, but not the 2-to-1 margin needed for consensus.

This paper, P1227, was a proposal to add non-member std::ssize and member ssize() functions. The inclusion of these would make certain code much more straightforward and allow for the avoidance of unwanted unsigned-ness in size computations. The idea was that the resistance to P1089 would decrease if ssize() were made available for all containers, both through std::ssize() and as member functions.

P1089 and P1227 were discussed at length during an evening session in San Diego 2018. The next day, a new poll was taken during a special joint session between EWG and LEWG. "Proposal 4" received the highest amount of approval, however since it was not an actual paper, and had no actual author, it's not entirely clear what it was. This paper attempts to document what I believe "Proposal 4" was.

I believe that the objection to "P1089+P1227" was that P1227 had a clause stating that an ssize() member function should be added to all STL containers; this implied by extension that all containers should have ssize() member functions, and the amount of work to do that was strongly resisted by some in attendance. I do not believe that any part of P1089 caused objection.

In a nutshell: Every part of span that used signed values to represent a size or an index will be converted to use unsigned, except for "difference_type". Span's signed size() function will be renamed ssize() in order to adopt a size() function that returns a value of type size_t, like every other STL container. std::ssize(container) is a new function the same value as same as std::size(container), but in a signed type.

I. Motivation and Scope.

For motivation to use the unsigned size_t type, see <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1089r2.pdf>

For motivation to use a signed size type, consider the following code:

```
template <typename T>
bool has_repeated_values(const T& container) {
    for (int i = 0; i < container.size() - 1; ++i) {
        if (container[i] == container[i + 1]) return true;
    }
    return false;
}
```

An experienced C++ programmer would immediately see the problem here, but programmers new to the language often make the mistake seen here: subtraction of 1 from a size of zero does not produce a negative value, but rather, produces a very very large positive value. So when this routine is called on an empty container, undefined behavior results.

This has been discussed ad infinitum, most recently at the standards level at the Rapperswil 2018 LEWG meeting, during a discussion of spans and ranges; the suggestion was made to put forth a proposal to explicitly add `ssize()` member functions to all STL containers, and to add a non-member `std::ssize()` function. Once these exist, programmers can simply write:

```
template <typename T>
bool has_repeated_values(const T& container) {
    for (ptrdiff_t i = 0; i < container.ssize() - 1; ++i) {
        if (container[i] == container[i + 1]) return true;
    }
    return false;
}
```

At 2018 San Diego, the `ssize()` member functions did not have as much consensus as a proposal without them, and were removed. Therefore the code is:

```
template <typename T>
bool has_repeated_values(const T& container) {
    using std::ssize; // needed iff container is not an STL container.
    for (ptrdiff_t i = 0; i < ssize(container) - 1; ++i) {
        if (container[i] == container[i + 1]) return true;
    }
    return false;
}
```

Impact on the Standard

This proposal adds a pure library extension that could be implemented in C++11, and alters the specification of `std::span`, slated to be part of C++20, but not yet published.

Proposed Wording

Note: All changes are relative to N4800.

Modify 21.7.2 [span.syn] :

```
inline constexpr ptrdiff_t size_t dynamic_extent = 1numeric_limits<size_t>::max();
template <class ElementType, ptrdiff_t size_t Extent = dynamic_extent>
class span;

template <class ElementType, ptrdiff_t size_t Extent>
span<const byte,
Extent == dynamic_extent ? dynamic_extent
: static_cast<ptrdiff_t>(sizeof(ElementType)) * Extent>
as_bytes(span<ElementType, Extent> s) noexcept;
template <class ElementType, ptrdiff_t size_t Extent>
span<byte,
Extent == dynamic_extent ? dynamic_extent
: static_cast<ptrdiff_t>(sizeof(ElementType)) * Extent>
as_writable_bytes(span<ElementType, Extent> s) noexcept;
```

Change span synopsis [span.overview]

~~3 If Extent is negative and not equal to dynamic_extent, the program is ill-formed.~~

```
template <class ElementType, ptrdiff_t size_t Extent = dynamic_extent>
class span {
    using index_type = ptrdiff_t size_t;
    template <class OtherElementType, ptrdiff_t size_t OtherExtent>
```

```
constexpr span(const span<OtherElementType, OtherExtent>& s)
noexcept;
template <ptrdiff_t size_t Count>
constexpr span<element_type, Count> first() const;
template <ptrdiff_t size_t Count>
constexpr span<element_type, Count> last() const;
template <ptrdiff_t size_t Offset, ptrdiff_t size_t Count =
dynamic_extent>
constexpr span<element_type, see below > subspan() const;
```

Change span "Constructors, copy, and assignment" [span.cons]

```
template<class OtherElementType, ptrdiff_t size_t OtherExtent>
constexpr span(const span<OtherElementType, OtherExtent>& s) noexcept;
```

Change [span.sub]

```
template <ptrdiff_t size_t Count> constexpr span<element_type, Count>
first() const;
1. Requires: 0 <= Count && Count <= size().

template <ptrdiff_t size_t Count> constexpr span<element_type, Count>
last() const;

3. Requires: 0 <= Count && Count <= size().

template <ptrdiff_t size_t Offset, ptrdiff_t size_t Count = dynamic_extent>
constexpr span<element_type, see below > subspan() const;

5. Requires:
(0 <= Count && Offset <= size()) &&
(Count == dynamic_extent || Count >= 0 && Offset + Count <= size())

8. Requires: 0 <= count && count <= size().

10. Requires: 0 <= count && count <= size().

12. Requires: (0 <= offset && offset <= size())
&& (count == dynamic_extent || count >= 0 && offset + count <= size())
```

Change [span.elem]

```
1. Requires: 0 <= idx && idx < size().
```

Change [span.objectrep]

```
template <class ElementType, ptrdiff_t size_t Extent>
span<const byte, Extent == dynamic_extent ? dynamic_extent
: static_cast<ptrdiff_t>(sizeof(ElementType)) * Extent>
as_bytes(span<ElementType, Extent> s) noexcept;

template <class ElementType, ptrdiff_t size_t Extent>
span<byte, Extent == dynamic_extent ? dynamic_extent
: static_cast<ptrdiff_t>(sizeof(ElementType)) * Extent>
as_writable_bytes(span<ElementType, Extent> s) noexcept;
```

Proposed Additional Text

Modify the section 27.3 Header synopsis [iterator.synopsis] by adding the following near the declarations for size():

```
template <class C>
constexpr auto ssize(const C& c)
-> common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>;

template <class T, ptrdiff_t N>
constexpr ptrdiff_t ssize(const T (&array)[N]) noexcept;
```

Modify the section 27.8 Iterator Range access by adding the following near the definitions for size():

```
template <class C>
constexpr auto ssize(const C& c);
    // Returns: static_cast<common_type_t<ptrdiff_t, make_signed_t<decltype(c.size())>>>(c.size())

template <class T, ptrdiff_t N>
constexpr ptrdiff_t ssize(const T (&array)[N]) noexcept;
    // Returns: N.
```

Note that the code does not just return the std::make_signed variant of the container's size() method, because it's conceivable that a container might choose to represent its size as a uint16_t, supporting up to 65,535 elements, and it would be a disaster for std::ssize() to turn a size of 60,000 into a size of -5,536.

Note also that the type of the array size is ptrdiff_t, not size_t. This is intentional: in a conceivable embedded environment where an array has size larger than ptrdiff_t can hold, but within the range of size_t, calling ssize() on it will cause a compile error due to template substitution failure. See [this godbolt example](#).

References

- Nevin Liber, Robert Douglas, Marshall Clow, [Sizes Should Only span Unsigned \(P1089R2\)](#)
- Jorg Brown, [Signed ssize\(\) functions, unsigned size\(\) functions \(P1227R0\)](#)
- Jorg Brown, [Signed ssize\(\) functions, unsigned size\(\) functions \(P1227R1\)](#)